PDF-версия исходной лекции iSMART из сохранённого HTML. Содержание не заменено пересказом. Служебные элементы интерфейса исключены.

Полный цикл ML-проекта на датасете

Содержание

- 1. Чему вы научитесь
- 2. Среда выполнения и правила работы
- 3. Краткий теоретический минимум
- 4. Чек-лист перед сдачей

Вступление

Привет! На прошлых занятиях вы уже знакомились с отдельными элементами машинного обучения: данными, признаками, моделями и метриками. Сегодня вы соберёте их в единую рабочую цепочку — так, как это делают аналитики и ML-разработчики. Вы получите датасет, сформулируете задачу, проверите данные, разделите их, обучите модель, проведёте валидацию и только после этого оцените качество на тестовой выборке. К концу занятия у вас будет мини-проект, который можно запустить повторно и объяснить другому человеку.
Главная трудность ML-проекта обычно заключается не в одном вызове метода обучения, а в правильной организации всех этапов. Даже подходящая модель может показать недостоверный результат, если данные разделены некорректно, масштабирование выполнено до разбиения или тестовая выборка использовалась при подборе настроек. Поэтому на занятии важно не только получить метрики, но и уметь обосновать порядок действий.
Рабочая цепочка должна быть воспроизводимой: повторный запуск ноутбука сверху вниз приводит к тому же способу подготовки данных, тому же разбиению и сопоставимым результатам. Это позволит проверить проект и отличить закономерный результат от случайно удачного эксперимента.

1. Чему вы научитесь

- различать объект, признак и целевую переменную;

- проверять размер, типы данных, пропуски, дубликаты и баланс классов;
- разделять данные на обучающую и тестовую части без утечки информации;
- объединять масштабирование и модель в Pipeline;
- подбирать гиперпараметр по кросс-валидации;
- рассчитывать accuracy, macro precision, macro recall и macro F1;
- читать матрицу ошибок и формулировать аргументированный вывод о качестве модели.

Эти умения образуют единый процесс. Например, значение метрики нельзя правильно интерпретировать отдельно от способа разбиения данных, а сравнивать модели можно только при одинаковых условиях проверки.

Результат занятия
Один запускаемый ноутбук с кодом и мини-отчёт из 6–8 предложений: что предсказывала модель, как были подготовлены данные, как выполнялась валидация, какие метрики получены, где модель ошиблась и можно ли считать результат приемлемым.

Мини-отчёт должен опираться на полученные значения, а не только на общую оценку «модель работает хорошо». Укажите хотя бы одну числовую метрику, назовите наиболее заметное направление ошибок по матрице ошибок и обозначьте ограничение проведённой проверки.

2. Среда выполнения и правила работы

Используйте встроенную среду учебной платформы. Резервные варианты — Google Colab или локальный Jupyter Notebook. Для локальной установки библиотек можно выполнить:

```
python -m pip install pandas matplotlib scikit-learn
```

Команда устанавливает библиотеки в среду Python, из которой она была запущена. Если Jupyter Notebook использует другое окружение, после установки импорт может не сработать. В таком случае необходимо проверить, что команда установки и ядро ноутбука относятся к одному окружению, а затем перезапустить ядро.

- Выполняйте ячейки сверху вниз. После изменения ранней ячейки повторно запустите зависимые ячейки.
- Не подглядывайте в тестовую выборку при выборе модели или гиперпараметров.
- Не удаляйте random_state=42: он делает разбиение воспроизводимым.
- Сохраняйте выводы рядом с кодом. Число без объяснения не считается анализом.

- Перед сдачей перезапустите среду и выполните все ячейки заново сверху вниз. Так можно обнаружить скрытую зависимость от переменной или результата, оставшегося от предыдущего запуска.
- Сравнивайте варианты модели на одном и том же разбиении и по одной схеме кросс-валидации. Иначе различие метрик может объясняться не моделями, а разными условиями проверки.

Важно. Фиксация random_state обеспечивает повторяемость случайных операций, но сама по себе не гарантирует качество модели. Это техническое условие честного сравнения экспериментов, а не способ улучшить метрики.

3. Краткий теоретический минимум

3.1. Полный цикл ML-проекта

Модель — только один элемент проекта. Надёжный результат появляется, когда каждый шаг цепочки выполнен осмысленно и воспроизводимо.
Сначала формулируют задачу: определяют, что является объектом, какие столбцы служат признаками и какую целевую переменную требуется предсказать. Затем исследуют данные, чтобы обнаружить пропуски, дубликаты, неподходящие типы и возможный дисбаланс классов. Только после этих проверок данные разделяют, на обучающей части подбирают способ обработки и настройки модели, а тестовую часть сохраняют для финальной оценки.
Порядок этапов важен. Если сначала использовать весь датасет для вычисления параметров предобработки, а затем провести разбиение, информация о тестовых объектах косвенно попадёт в обучение. Такая ситуация называется утечкой данных и делает итоговую оценку слишком оптимистичной.
Воспроизводимость означает, что в ноутбуке зафиксированы не только алгоритм и его настройки, но и последовательность преобразований. Другой человек должен иметь возможность запустить проект и понять, из каких данных и действий получен результат.
Рисунок 1. Мини-пайплайн машинного обучения

3.2. Термины занятия

Термин	Простое объяснение
Объект	Одна строка датасета — один пример, для которого известны признаки.

Термин	Простое объяснение
Признак X	Измеряемая характеристика объекта, используемая моделью для прогноза.
Целевая переменная y	Правильная метка, которую модель должна научиться предсказывать.
Классификация	Задача выбора одного класса из конечного набора классов.
Гиперпараметр	Настройка, задаваемая до обучения; в KNN это число соседей k.
Валидация	Оценивание нескольких вариантов модели внутри обучающей части.
Тестовая выборка	Отложенные данные для финальной проверки выбранного решения.
Pipeline	Единая цепочка предобработки и модели, уменьшающая риск ошибок и утечки данных.

Например, если датасет содержит измерения нескольких объектов и метку их класса, одна строка является объектом, столбцы с измерениями образуют матрицу признаков X, а столбец с классом — целевую переменную y. Целевой столбец нельзя оставлять среди признаков: тогда модель получит правильный ответ на входе, и оценка потеряет смысл.
Гиперпараметр следует отличать от параметров, которые модель получает в процессе обучения. Значение k для KNN выбирается до финального обучения, тогда как необходимые сведения об обучающих объектах алгоритм получает при вызове обучения.
Pipeline можно представить как закреплённую последовательность действий: входные данные сначала проходят преобразование StandardScaler, а затем поступают в KNN. При предсказании для новых объектов используется тот же порядок и те параметры масштабирования, которые были вычислены на обучающих данных.
Обратите внимание. Тестовая выборка не является ещё одной валидационной частью. Если результат на тесте повлиял на выбор k, набора признаков или модели, тест уже участвовал в разработке решения и перестал быть независимым.

3.3. Почему нельзя выбирать модель по тесту

Тестовая выборка играет роль независимого экзамена. Если много раз смотреть на тестовые метрики и подстраивать модель, тест перестаёт быть независимым. Поэтому гиперпараметр выбирают по кросс-валидации на обучающей части, затем один раз оценивают выбранную модель на тесте.

При кросс-валидации обучающая часть несколько раз делится на внутренние обучающие и валидационные фрагменты. Модель обучается на одних фрагментах и проверяется на другом, причём роль валидационного фрагмента по очереди выполняют разные части данных. Это позволяет оценить не только средний результат, но и его устойчивость к конкретному разбиению.

Например, при 5-кратной кросс-валидации получают пять значений метрики. Для выбора k можно сравнить их средние значения, а разброс использовать как дополнительный признак стабильности. При этом тестовые объекты не должны попадать ни в один из пяти этапов.

После выбора гиперпараметра модель с выбранными настройками обучают на всей обучающей части. Затем выполняют единственную финальную проверку на тестовой выборке. Если после этого изменить решение на основании тестовой метрики, для действительно независимой оценки потребуется новый набор данных.

Рисунок 2. Разделение ролей обучающих, валидационных и тестовых данных

3.4. Метрики

Метрика	Что показывает	На что обратить внимание
Accuracy	Доля всех правильных прогнозов.	Удобна при примерно сопоставимых классах.
Precision	Какая доля объектов, названных моделью классом, действительно относится к нему.	Важна, когда ложное срабатывание дорого.
Recall	Какую долю реальных объектов класса модель нашла.	Важна, когда опасно пропустить объект.
F1	Гармоническое среднее precision и recall.	Полезна для компромисса между двумя типами ошибок.
Матрица ошибок	Сколько объектов каждого истинного класса попало в каждый предсказанный класс.	Показывает не только число, но и направление ошибок.

Accuracy рассчитывается как отношение числа правильных прогнозов к общему числу объектов. Например, если модель правильно классифицировала 90 объектов из 100, accuracy равна 0,90. Однако одна и та же accuracy может скрывать разные ошибки: модель могла понемногу ошибаться во всех классах или почти полностью игнорировать один редкий класс.

Precision и recall рассматривают качество с разных сторон. Высокая precision означает, что среди объектов, которым модель назначила некоторый класс, мало ошибочных назначений. Высокая recall означает, что модель обнаружила большую долю всех реальных объектов этого класса. Улучшение одной из этих характеристик не всегда автоматически улучшает другую.

F1 объединяет precision и recall и принимает высокое значение только тогда, когда обе метрики достаточно высоки. Поэтому F1 полезно анализировать вместе с accuracy, особенно если размеры классов различаются.

В многоклассовой задаче macro-усреднение сначала считает метрику отдельно для каждого класса, а затем берёт обычное среднее. Каждый класс получает одинаковый вес.

Например, при трёх классах macro recall равна среднему арифметическому recall для первого, второго и третьего классов. Поэтому слабый результат даже на небольшом классе заметно уменьшит итоговую macro-метрику. В этом состоит отличие macro-усреднения от показателей, в которых вклад класса зависит от числа его объектов.

В матрице ошибок строки обычно соответствуют истинным классам, а столбцы — предсказанным. Элементы главной диагонали показывают правильные ответы, а элементы вне диагонали — ошибки. Например, элемент **cm[1, 2]** показывает, сколько объектов истинного класса 1 было предсказано как класс 2.

Важно. При чтении матрицы ошибок необходимо проверить порядок меток классов, использованный функцией. Индексы строки и столбца относятся к позициям классов в этом порядке и не всегда совпадают с их смысловыми названиями.

Мини-проверка: если accuracy высока, но macro recall заметно ниже, следует проверить, не пропускает ли модель значительную часть объектов одного или нескольких классов.

4. Чек-лист перед сдачей

☐ Я сформулировал тип задачи и целевую переменную.
☐ Я проверил размер, пропуски, дубликаты и баланс классов.
☐ Я разделил данные до масштабирования.
☐ Я объединил StandardScaler и KNN в Pipeline.
☐ Я выбрал k по кросс-валидации на обучающей части.
☐ Я использовал тестовую часть только для финальной оценки.
☐ Я посчитал не только accuracy, но и macro F1.
☐ Я объяснил матрицу ошибок и влияние масштабирования.
☐ Мой вывод содержит числа, ограничения и аргументацию.
☐ Я сравнивал варианты модели на одинаковых разбиениях.
☐ Я проверил порядок классов в матрице ошибок.
☐ После перезапуска среды все ячейки выполняются сверху вниз без ошибок.

Если хотя бы один пункт, связанный с разделением данных или использованием тестовой выборки, не выполнен, сначала исправьте схему эксперимента, а затем пересчитайте метрики. Старые значения после такого изменения использовать нельзя.

Справочные материалы

Официальная документация scikit-learn: load_wine, Pipeline, cross_val_score и раздел Classification metrics. Адрес: https://scikit-learn.org/stable/
При работе с документацией обращайте внимание на описание параметров функций, формат возвращаемых значений и примеры использования. Для метрик особенно важно проверять параметры усреднения, порядок меток классов и то, какие массивы передаются как истинные и предсказанные ответы.

Датасет практикума

Содержание

- 1. Практическая часть
- 3. Дополнительные задания

Используется встроенный набор Wine из scikit-learn. Он содержит 178 объектов, 13 числовых признаков и 3 класса. Датасет учебный: классы обозначены class_0, class_1 и class_2.
Каждая строка набора соответствует одному объекту, а каждый столбец признаков — отдельному числовому измерению этого объекта. Модель должна по значениям 13 признаков определить, к какому из трёх классов относится объект. Поскольку возможных классов больше двух, решается задача многоклассовой классификации.
Названия class_0, class_1 и class_2 являются метками категорий. Числа 0, 1 и 2 в целевой переменной используются как их машинное представление и не означают порядок или степень качества.

Группа признаков	Примеры
Химический состав	alcohol, malic_acid, magnesium, total_phenols, flavanoids
Цвет и оптические характеристики	color_intensity, hue, od280/od315_of_diluted_wines
Другие измерения	ash, alcalinity_of_ash, proanthocyanins, proline

Группировка помогает понять смысл столбцов, но сама по себе не изменяет данные: при обучении каждый из перечисленных показателей остаётся отдельным признаком. Все признаки числовые, однако их значения могут иметь разные диапазоны, что особенно важно для методов, основанных на расстояниях.

> **Важно**
>
> Цель занятия — освоить технологию ML-проекта, а не делать выводы о свойствах реальных напитков. Не используйте учебную модель для реальных решений.

Учебный результат относится только к данному набору данных и выбранной схеме проверки. Высокое значение метрики на тестовой части не доказывает, что модель будет столь же точной

на любых новых данных.

1. Практическая часть

Работайте последовательно. После каждой задачи выполните автопроверку и заполните краткий вывод.
Последовательность этапов важна: сначала формулируется задача и исследуются данные, затем данные разделяются, после этого выбираются преобразования и гиперпараметры, и только в конце проводится однократная оценка на тестовой выборке.

3. Дополнительные задания

Выполняются после основных задач и не снижают оценку, если не завершены.

Дополнительное задание 1. Сравнение с деревом решений

Сравните KNN с DecisionTreeClassifier(max_depth=3, random_state=42). Выбирайте модель по одинаковой схеме валидации.
При сравнении KNN и дерева решений сохраните одинаковое разбиение на обучающую и тестовую части, одинаковые folds кросс-валидации и одинаковый набор метрик. Это делает сравнение корректным. Учтите, что масштабирование существенно для KNN, поскольку алгоритм использует расстояния между объектами; для дерева решений оно обычно не меняет правила разбиения.

Код для выполнения

```
from sklearn.tree import DecisionTreeClassifier

tree = DecisionTreeClassifier(max_depth=3, random_state=42)
tree_pipeline = Pipeline([
    ("scale", StandardScaler()),
    ("model", tree)
])

tree_scores = cross_val_score(
    tree_pipeline,
    X_train,
    y_train,
    cv=cv,
    scoring="accuracy"
)
print("Tree CV mean:", round(tree_scores.mean(), 4))
print("Tree CV std:", round(tree_scores.std(), 4))

tree_pipeline.fit(X_train, y_train)
tree_pred = tree_pipeline.predict(X_test)
print("Tree test accuracy:", round(accuracy_score(y_test, tree_pred), 4))
```

Сравнение результатов

Модель	CV mean	CV std	Test accuracy
KNN (k=3)			
Decision Tree (max_depth=3)			

Какая модель показала лучший результат? Почему?

_____

_____

Дополнительное задание 2. Отбор признаков по дисперсии

Оставьте только 5 признаков с наибольшей дисперсией. Проверьте, как изменятся CV и test metrics.

При отборе признаков не используйте тестовую часть для выбора пяти столбцов. Их дисперсию следует оценивать по обучающим данным, иначе сведения о тестовом наборе повлияют на подготовку модели. После отбора сравните не только среднее CV, но и изменение разброса результатов.

Код для выполнения

```
variances = X_train.var().sort_values(ascending=False)
top_5_features = variances.head(5).index.tolist()
print("Отобранные признаки:", top_5_features)

X_train_subset = X_train[top_5_features]
X_test_subset = X_test[top_5_features]

model_subset = Pipeline([
    ("scale", StandardScaler()),
    ("model", KNeighborsClassifier(n_neighbors=best_k))
])

subset_scores = cross_val_score(
    model_subset,
    X_train_subset,
    y_train,
    cv=cv,
    scoring="accuracy"
)
print("Subset CV mean:", round(subset_scores.mean(), 4))
print("Subset CV std:", round(subset_scores.std(), 4))

model_subset.fit(X_train_subset, y_train)
subset_pred = model_subset.predict(X_test_subset)
print("Subset test accuracy:", round(accuracy_score(y_test, subset_pred), 4))
```

Результаты

Отобранные признаки: _____

Модель	CV mean	CV std	Test accuracy
Все признаки			

Модель	CV mean	CV std	Test accuracy
Только 5 признаков			

Изменилось ли качество модели? Почему?

_____

_____

Дополнительное задание 3. Анализ ошибочных объектов

Найдите индексы ошибочно классифицированных объектов и сравните их признаки со средними значениями истинного и предсказанного классов.
При анализе ошибочных объектов отдельно укажите истинный и предсказанный классы.
Сравнение признаков не доказывает причину ошибки, но помогает сформулировать проверяемое предположение: например, ошибочный объект может быть ближе по значениям некоторых признаков к средним значениям предсказанного класса.

Код для выполнения

```
error_indices = np.where(y_pred != y_test)[0]
print("Индексы ошибок в X_test:", error_indices)

if len(error_indices) > 0:
    idx = error_indices[0]
    print(f"\nПример ошибки {idx}:")
    print(f"Истинный класс: {y_test.iloc[idx]}")
    print(f"Предсказанный класс: {y_pred[idx]}")
    print("Значения признаков ошибочного объекта:")
    print(X_test.iloc[idx].round(3))

    print("\nСредние значения для истинного класса:")
    print(X_train[y_train == y_test.iloc[idx]].mean().round(3))

    print("\nСредние значения для предсказанного класса:")
    print(X_train[y_train == y_pred[idx]].mean().round(3))
```

Анализ

Индексы ошибочных объектов: _____

Для первой ошибки:

Истинный класс: _____ Предсказанный класс: _____

Какие признаки отличаются от средних значений классов больше всего?

_____

_____

Какое предположение о причине ошибки можно сформулировать?

_____

_____